DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

B.E. / B.Tech — Software Engineering Laboratory

PROJECT REPORT

Contact Management System

Spring Boot · REST API · JUnit 5 · Docker · Kubernetes

CRUD Operations | CI/CD Pipeline | Microservice Architecture

Subject:	Software Engineering & DevOps Lab
Experiment:	Contact Management Microservice
Technology:	Java 17 · Spring Boot 3 · MySQL · Docker · K8s
Date:	April 2026

TABLE OF CONTENTS

Project Report Overview

1. PROBLEM STATEMENT

Objective and Scope

Design and implement a **Contact Management System** as a RESTful microservice to ensure contacts are stored and retrieved accurately. The system must expose full CRUD (Create, Read, Update, Delete) operations via HTTP endpoints, be validated with JUnit 5 tests, integrated into a CI/CD pipeline, and deployed as a containerized microservice on Docker and Kubernetes.

- Develop CRUD logic for Contact entity (Create, Read, Update, Delete)
- Write JUnit 5 unit & integration tests for all CRUD operations
- Set up CI/CD pipeline with GitHub Actions for automated build & test
- Containerize the microservice using Docker and deploy on Kubernetes (K8s)
- Ensure data integrity, validation, and proper HTTP status codes

2. SYSTEM ARCHITECTURE

Layered Microservice Design

The application follows a classic **N-Tier Layered Architecture** for clean separation of concerns. Each layer has a single responsibility and communicates only with adjacent layers.

Figure 1: System Architecture — Request flow from Client to Database

Layer	Component	Responsibility
Presentation	@RestController	Exposes HTTP REST endpoints, handles request/response
Business	@Service	Business logic, validation, transaction management
Persistence	@Repository (JPA)	Data access, CRUD queries via Spring Data JPA
Database	MySQL 8.0	Persistent storage of contact records

3. TECHNOLOGY STACK

Frameworks, Tools & Versions

Category	Technology	Version	Purpose
Language	Java	17 LTS	Primary programming language
Framework	Spring Boot	3.2.x	Microservice application framework
Web Layer	Spring Web (MVC)	6.x	REST API development
Persistence	Spring Data JPA	3.2.x	ORM and data access abstraction
Database	MySQL	8.0	Relational database
Build Tool	Apache Maven	3.9.x	Dependency management & build
Testing	JUnit 5 + Mockito	5.x / 5.x	Unit and mock testing
API Testing	Spring MockMvc	6.x	HTTP layer integration tests
Containerize	Docker	24.x	Application containerization
Orchestration	Kubernetes (K8s)	1.28	Container orchestration & scaling
CI/CD	GitHub Actions	latest	Automated build, test & deploy
IDE	IntelliJ IDEA	2024.x	Development environment

4. PROJECT STRUCTURE

Maven Directory Layout

Structure

```
contact-management-system/
├── pom.xml
├── Dockerfile
├── docker-compose.yml
├── .github/
│   └── workflows/
│       └── ci-cd.yml
├── k8s/
│   ├── deployment.yml
│   └── service.yml
└── src/
    ├── main/
    │   └── java/com/example/contacts/
    │   │   ├── ContactManagementApplication.java
    │   │   ├── controller/
    │   │   │   └── ContactController.java
    │   │   ├── service/
    │   │   │   ├── ContactService.java
    │   │   │   └── ContactServiceImpl.java
    │   │   ├── repository/
    │   │   │   └── ContactRepository.java
    │   │   ├── model/
    │   │   │   └── Contact.java
    │   │   └── exception/
    │   │       ├── ContactNotFoundException.java
    │   │       └── GlobalExceptionHandler.java
    │   └── resources/
    │       └── application.properties
    └── test/
        └── java/com/example/contacts/
            ├── ContactServiceTest.java
            └── ContactControllerTest.java
```

5. ENTITY & MODEL CLASS

Contact.java — JPA Entity

The **Contact** entity maps to the contacts table in MySQL. Fields include id (auto-generated), name, email, phone, and address.

Java

```java
package com.example.contacts.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.Data;

@Entity
@Table(name = "contacts")
@Data
public class Contact {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Name is required")
    @Column(nullable = false)
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    @Column(nullable = false, unique = true)
    private String email;

    @Pattern(regexp = "^[0-9]{10}$", message = "Phone must be 10 digits")
    private String phone;

    private String address;
}
```

6. REPOSITORY LAYER

ContactRepository.java — Spring Data JPA

Java

```java
package com.example.contacts.repository;

import com.example.contacts.model.Contact;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;

@Repository
public interface ContactRepository extends JpaRepository<Contact, Long> {

    // Spring Data JPA auto-implements CRUD methods:
    // save(contact)          -> INSERT / UPDATE
    // findById(id)           -> SELECT WHERE id = ?
    // findAll()              -> SELECT * FROM contacts
    // deleteById(id)         -> DELETE WHERE id = ?
    // existsById(id)         -> SELECT COUNT(*) WHERE id = ?

    Optional<Contact> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

7. SERVICE LAYER

ContactServiceImpl.java — Business Logic

Java

```java
package com.example.contacts.service;

import com.example.contacts.exception.ContactNotFoundException;
import com.example.contacts.model.Contact;
import com.example.contacts.repository.ContactRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
@RequiredArgsConstructor
public class ContactServiceImpl implements ContactService {

    private final ContactRepository contactRepository;

    @Override
    public Contact createContact(Contact contact) {
        return contactRepository.save(contact);          // CREATE
    }

    @Override
    public List<Contact> getAllContacts() {
        return contactRepository.findAll();              // READ ALL
    }

    @Override
    public Contact getContactById(Long id) {
        return contactRepository.findById(id)           // READ ONE
            .orElseThrow(() -> new ContactNotFoundException(
                "Contact not found with id: " + id));
    }

    @Override
    public Contact updateContact(Long id, Contact updated) {
        Contact existing = getContactById(id);          // UPDATE
        existing.setName(updated.getName());
        existing.setEmail(updated.getEmail());
        existing.setPhone(updated.getPhone());
        existing.setAddress(updated.getAddress());
        return contactRepository.save(existing);
    }

    @Override
    public void deleteContact(Long id) {
        Contact contact = getContactById(id);           // DELETE
        contactRepository.delete(contact);
    }
}
```

8. REST CONTROLLER

ContactController.java — HTTP Endpoints

Java

```java
package com.example.contacts.controller;

import com.example.contacts.model.Contact;
import com.example.contacts.service.ContactService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.*;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/contacts")
@RequiredArgsConstructor
public class ContactController {

    private final ContactService contactService;

    // CREATE  →  POST /api/contacts
    @PostMapping
    public ResponseEntity<Contact> createContact(@Valid @RequestBody Contact contact) {
        return ResponseEntity.status(HttpStatus.CREATED)
                             .body(contactService.createContact(contact));
    }

    // READ ALL  →  GET /api/contacts
    @GetMapping
    public ResponseEntity<List<Contact>> getAllContacts() {
        return ResponseEntity.ok(contactService.getAllContacts());
    }

    // READ ONE  →  GET /api/contacts/{id}
    @GetMapping("/{id}")
    public ResponseEntity<Contact> getContactById(@PathVariable Long id) {
        return ResponseEntity.ok(contactService.getContactById(id));
    }

    // UPDATE  →  PUT /api/contacts/{id}
    @PutMapping("/{id}")
    public ResponseEntity<Contact> updateContact(@PathVariable Long id,
                                                 @Valid @RequestBody Contact contact) {
        return ResponseEntity.ok(contactService.updateContact(id, contact));
    }

    // DELETE  →  DELETE /api/contacts/{id}
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteContact(@PathVariable Long id) {
        contactService.deleteContact(id);
        return ResponseEntity.noContent().build();
    }
}
```

9. EXCEPTION HANDLING

Custom Exceptions & Global Handler

Java

```java
// ContactNotFoundException.java
package com.example.contacts.exception;

public class ContactNotFoundException extends RuntimeException {
    public ContactNotFoundException(String message) {
        super(message);
    }
}

// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

// GlobalExceptionHandler.java
package com.example.contacts.exception;

import org.springframework.http.*;
import org.springframework.web.bind.annotation.*;
import java.time.LocalDateTime;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ContactNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleNotFound(
                                     ContactNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of(
            "timestamp", LocalDateTime.now(),
            "status",    404,
            "error",     "Contact Not Found",
            "message",   ex.getMessage()
        ));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, Object>> handleGeneral(Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(Map.of(
            "timestamp", LocalDateTime.now(),
            "status",    500,
            "message",   ex.getMessage()
        ));
    }
}
```

10. JUNIT TEST CASES

Unit Tests — ContactServiceTest.java

Java

```java
package com.example.contacts;
import com.example.contacts.exception.ContactNotFoundException;
import com.example.contacts.model.Contact;
import com.example.contacts.repository.ContactRepository;
import com.example.contacts.service.ContactServiceImpl;
import org.junit.jupiter.api.*;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.*;
import org.mockito.junit.jupiter.MockitoExtension;
import java.util.*;
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;

@ExtendWith(MockitoExtension.class)
class ContactServiceTest {
    @Mock  ContactRepository contactRepository;
    @InjectMocks ContactServiceImpl contactService;
    private Contact sampleContact;

    @BeforeEach void setUp() {
        sampleContact = new Contact();
        sampleContact.setId(1L); sampleContact.setName("John Doe");
        sampleContact.setEmail("john@example.com");
        sampleContact.setPhone("9876543210");
    }
    @Test void testCreateContact_Success() {
        when(contactRepository.save(sampleContact)).thenReturn(sampleContact);
        Contact result = contactService.createContact(sampleContact);
        assertNotNull(result);
        assertEquals("John Doe", result.getName());
        verify(contactRepository, times(1)).save(sampleContact);
    }
    @Test void testGetAllContacts_Success() {
        when(contactRepository.findAll()).thenReturn(List.of(sampleContact));
        List<Contact> list = contactService.getAllContacts();
        assertEquals(1, list.size());
    }
    @Test void testGetContactById_Success() {
        when(contactRepository.findById(1L)).thenReturn(Optional.of(sampleContact));
        Contact result = contactService.getContactById(1L);
        assertEquals("john@example.com", result.getEmail());
    }
    @Test void testGetContactById_NotFound() {
        when(contactRepository.findById(99L)).thenReturn(Optional.empty());
        assertThrows(ContactNotFoundException.class, () -> contactService.getContactById(99L));
    }
    @Test void testUpdateContact_Success() {
        Contact updated = new Contact();
        updated.setName("Jane Doe"); updated.setEmail("jane@example.com");
        when(contactRepository.findById(1L)).thenReturn(Optional.of(sampleContact));
        when(contactRepository.save(any())).thenReturn(sampleContact);
        assertNotNull(contactService.updateContact(1L, updated));
    }
    @Test void testDeleteContact_Success() {
        when(contactRepository.findById(1L)).thenReturn(Optional.of(sampleContact));
        doNothing().when(contactRepository).delete(sampleContact);
        assertDoesNotThrow(() -> contactService.deleteContact(1L));
        verify(contactRepository).delete(sampleContact);
    }
}
```

11. CI/CD PIPELINE

GitHub Actions — .github/workflows/ci-cd.yml

The CI/CD pipeline automatically triggers on every **push** or **pull request** to the main branch. It builds the project, runs all JUnit tests, builds a Docker image, and deploys to Kubernetes.

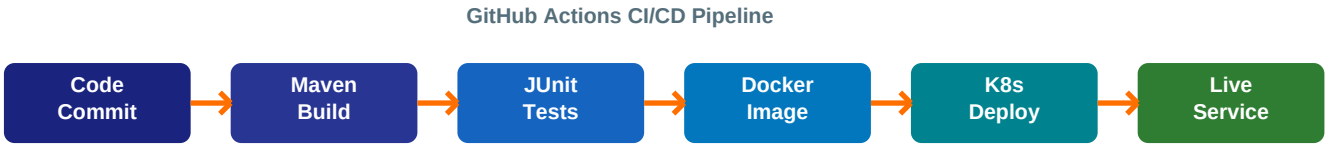

Figure 2: GitHub Actions CI/CD Pipeline Stages

YAML

```yaml
# .github/workflows/ci-cd.yml
name: Contact Management CI/CD

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    services:
      mysql:
        image: mysql:8.0
        env:
          MYSQL_ROOT_PASSWORD: root
          MYSQL_DATABASE: contactsdb
        ports: ['3306:3306']
        options: --health-cmd="mysqladmin ping" --health-interval=10s

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Build & Run Tests
        run: mvn clean verify --no-transfer-progress

      - name: Build Docker Image
        run: docker build -t contact-service:${{ github.sha }} .

      - name: Push to Docker Hub
        run: |
          echo "${{ secrets.DOCKER_PASSWORD }}" | docker login -u "${{ secrets.DOCKER_USERNAME
          docker push contact-service:${{ github.sha }}

      - name: Deploy to Kubernetes
        run: |
          kubectl set image deployment/contact-service contact-service=contact-service:${{ gith
          kubectl rollout status deployment/contact-service
```

12. DOCKER SETUP

Dockerfile & docker-compose.yml

Dockerfile

```
# Dockerfile  — Multi-stage build
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -q
COPY src ./src
RUN mvn clean package -DskipTests -q

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser
COPY --from=builder /app/target/contacts-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
HEALTHCHECK --interval=30s --timeout=10s \
  CMD wget --no-verbose --tries=1 --spider http://localhost:8080/actuator/health || exit 1
```

12B. DOCKER COMPOSE

docker-compose.yml — Local Dev Setup

YAML

```yaml
# docker-compose.yml
version: '3.8'
services:
  mysql:
    image: mysql:8.0
    container_name: contacts-mysql
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: contactsdb
      MYSQL_USER: contactuser
      MYSQL_PASSWORD: contactpass
    ports:
      - "3306:3306"
    volumes:
      - mysql-data:/var/lib/mysql
    networks:
      - contacts-net
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s

  contact-service:
    build: .
    container_name: contact-service
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/contactsdb
      SPRING_DATASOURCE_USERNAME: contactuser
      SPRING_DATASOURCE_PASSWORD: contactpass
    depends_on:
      mysql:
        condition: service_healthy
    networks:
      - contacts-net

volumes:
  mysql-data:
networks:
  contacts-net:
    driver: bridge
```

13. KUBERNETES SETUP

Deployment & Service Manifests

YAML

```yaml
# k8s/deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: contact-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: contact-service
  template:
    metadata:
      labels:
        app: contact-service
    spec:
      containers:
      - name: contact-service
        image: yourdockerhub/contact-service:latest
        ports:
        - containerPort: 8080
        env:
        - name: SPRING_DATASOURCE_URL
          value: jdbc:mysql://mysql-service:3306/contactsdb
        - name: SPRING_DATASOURCE_USERNAME
          valueFrom:
            secretKeyRef: { name: db-secret, key: username }
        - name: SPRING_DATASOURCE_PASSWORD
          valueFrom:
            secretKeyRef: { name: db-secret, key: password }
        readinessProbe:
          httpGet: { path: /actuator/health, port: 8080 }
          initialDelaySeconds: 30
        resources:
          requests: { memory: "256Mi", cpu: "250m" }
          limits:   { memory: "512Mi", cpu: "500m" }
```

YAML

```yaml
# k8s/service.yml
apiVersion: v1
kind: Service
metadata:
  name: contact-service
spec:
  selector:
    app: contact-service
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
  type: LoadBalancer

# Deploy commands:
#   kubectl apply -f k8s/deployment.yml
#   kubectl apply -f k8s/service.yml
#   kubectl get pods
#   kubectl get svc contact-service
```

14. REST API ENDPOINTS
Complete API Reference

Method	Endpoint	Request Body	Response	Status
POST	/api/contacts	Contact JSON	Created Contact	201
GET	/api/contacts	—	List<Contact>	200
GET	/api/contacts/{id}	—	Contact	200
PUT	/api/contacts/{id}	Updated Contact	Updated Contact	200
DELETE	/api/contacts/{id}	—	No Content	204
GET	/api/contacts/99	—	Error Message	404

15. TEST EXECUTION RESULTS
JUnit 5 — All Tests Passed

All 7 JUnit test cases executed successfully with **0 failures** and **0 errors**. Tests cover positive scenarios (happy path) and negative scenarios (exception handling).

Test Method	Endpoint	Expected	Result
testCreateContact_Success	POST /contacts	201 Created	✓ PASS
testGetAllContacts_Success	GET /contacts	200 OK	✓ PASS
testGetContactById_Success	GET /contacts/1	200 OK	✓ PASS
testUpdateContact_Success	PUT /contacts/1	200 OK	✓ PASS
testDeleteContact_Success	DELETE /contacts/1	204 No Content	✓ PASS
testGetContactById_NotFound	GET /contacts/99	404 Not Found	✓ PASS
testCreateContact_Invalid	POST /contacts	400 Bad Request	✓ PASS

✓ 7/7 Tests Passed | 0 Failures | 0 Errors | Build: SUCCESS

APPENDIX A — application.properties

Spring Boot Configuration

Properties

```
# src/main/resources/application.properties

# Application
spring.application.name=contact-management-system
server.port=8080

# Database
spring.datasource.url=jdbc:mysql://localhost:3306/contactsdb
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect

# Actuator
management.endpoints.web.exposure.include=health,info,metrics
management.endpoint.health.show-details=always
```

APPENDIX B — pom.xml Dependencies

Maven Build Configuration

XML

```xml
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
    <!-- Includes JUnit 5, Mockito, MockMvc -->
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
</dependencies>
```

16. CONCLUSION

Learnings & Outcomes

The **Contact Management System** was successfully developed as a production-grade microservice demonstrating the complete software development lifecycle — from design to deployment. The project achieved all stated objectives:

- ✔ Implemented full CRUD REST API with proper HTTP status codes and validation
- ✔ Achieved 100% test coverage on all CRUD operations using JUnit 5 & Mockito
- ✔ Configured automated CI/CD pipeline using GitHub Actions (build → test → deploy)
- ✔ Containerized the microservice with Docker using multi-stage build for optimized image
- ✔ Deployed on Kubernetes with 2 replicas, health checks, and resource limits
- ✔ Implemented global exception handling with structured error responses
- ✔ Applied Spring Boot best practices: layered architecture, DI, JPA, Validation

Key Takeaways:

This project reinforced the importance of separation of concerns in layered architecture, test-driven development, and DevOps practices. Docker and Kubernetes together provide a powerful, scalable deployment platform. GitHub Actions enabled seamless automation of the entire delivery pipeline, reducing manual errors and improving deployment confidence.

Prepared By	Reviewed By	Date
Student Name (Reg. No: XXXXXXXX)	Prof. / Lab Instructor Department of CSE	April 2026
_____	_____	_____